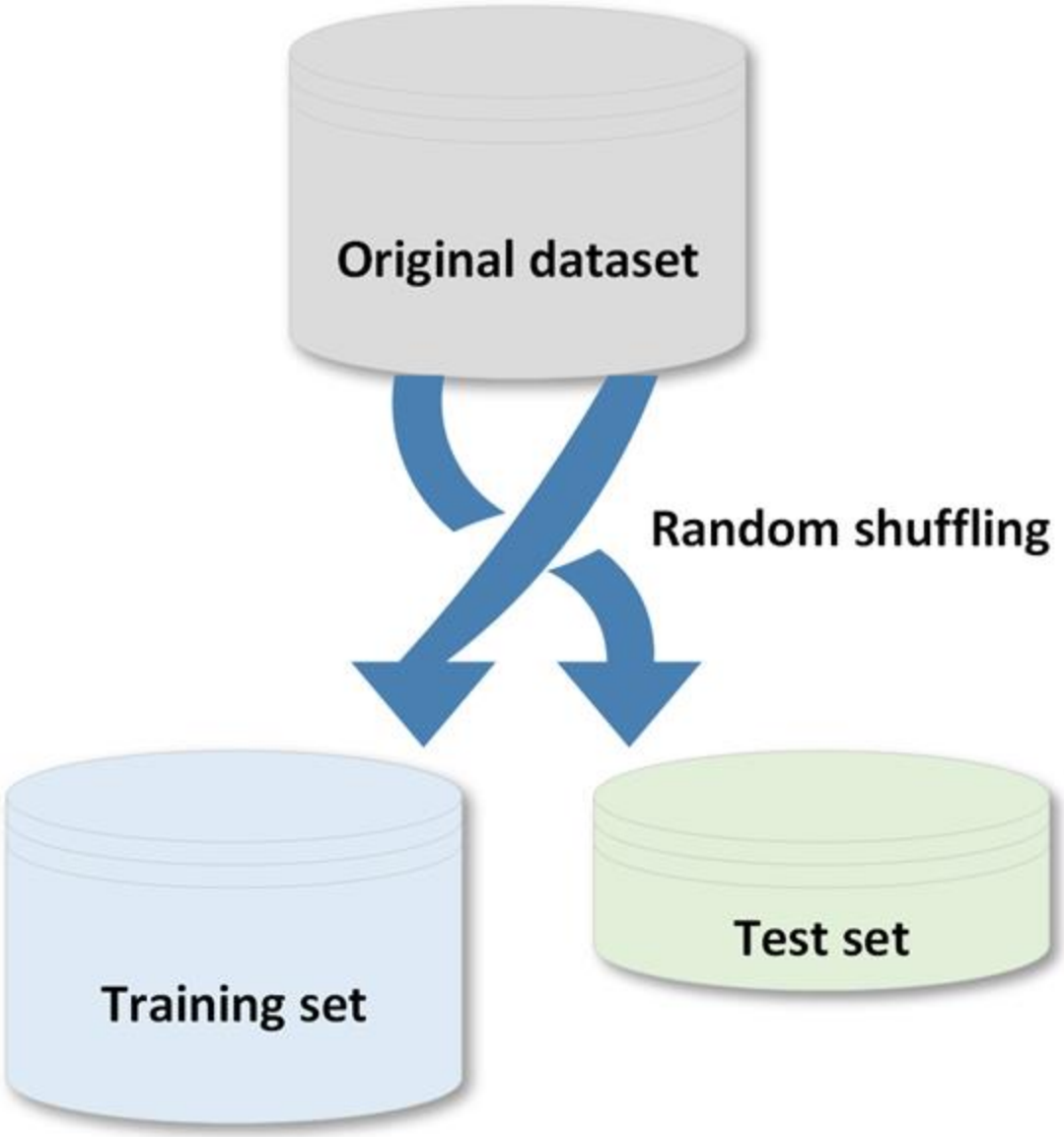
MLA-SESSION-II

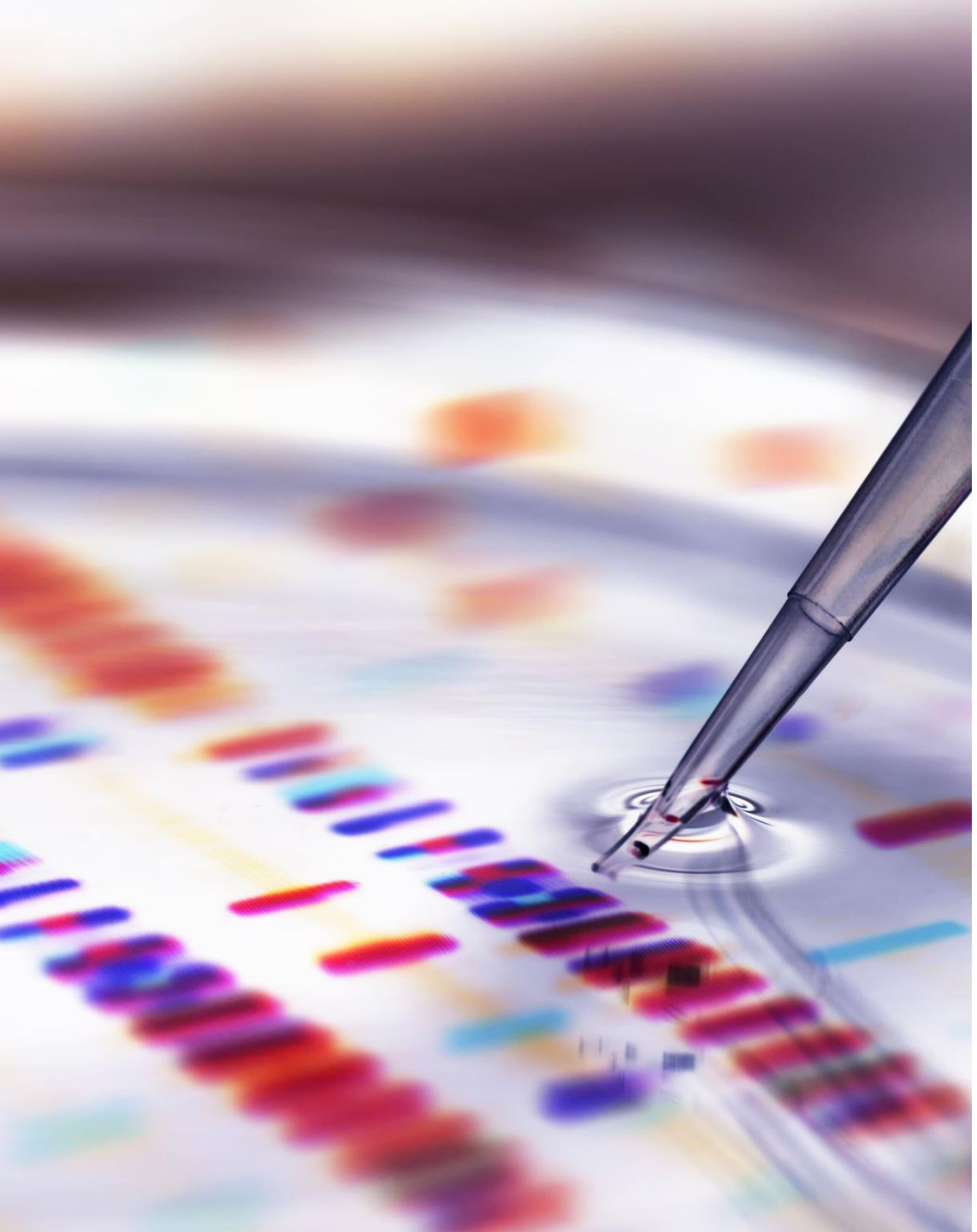
Training and Testing Class



Training Data:

Training Data:

- Training data is used to **teach the machine learning model** how to make **predictions or perform a desired task.**
- The model learns to **identify the patterns** in the data and make predictions based on those patterns.



.....

Testing Data

- **T**esting data is used to **evaluate the machine learning model's** performance.
- It is typically different from the training data and **not labeled**.
- This means the model's **output is unknown for each data point**.
- The model's ability to make **accurate predictions** on the testing data is **evaluated.**

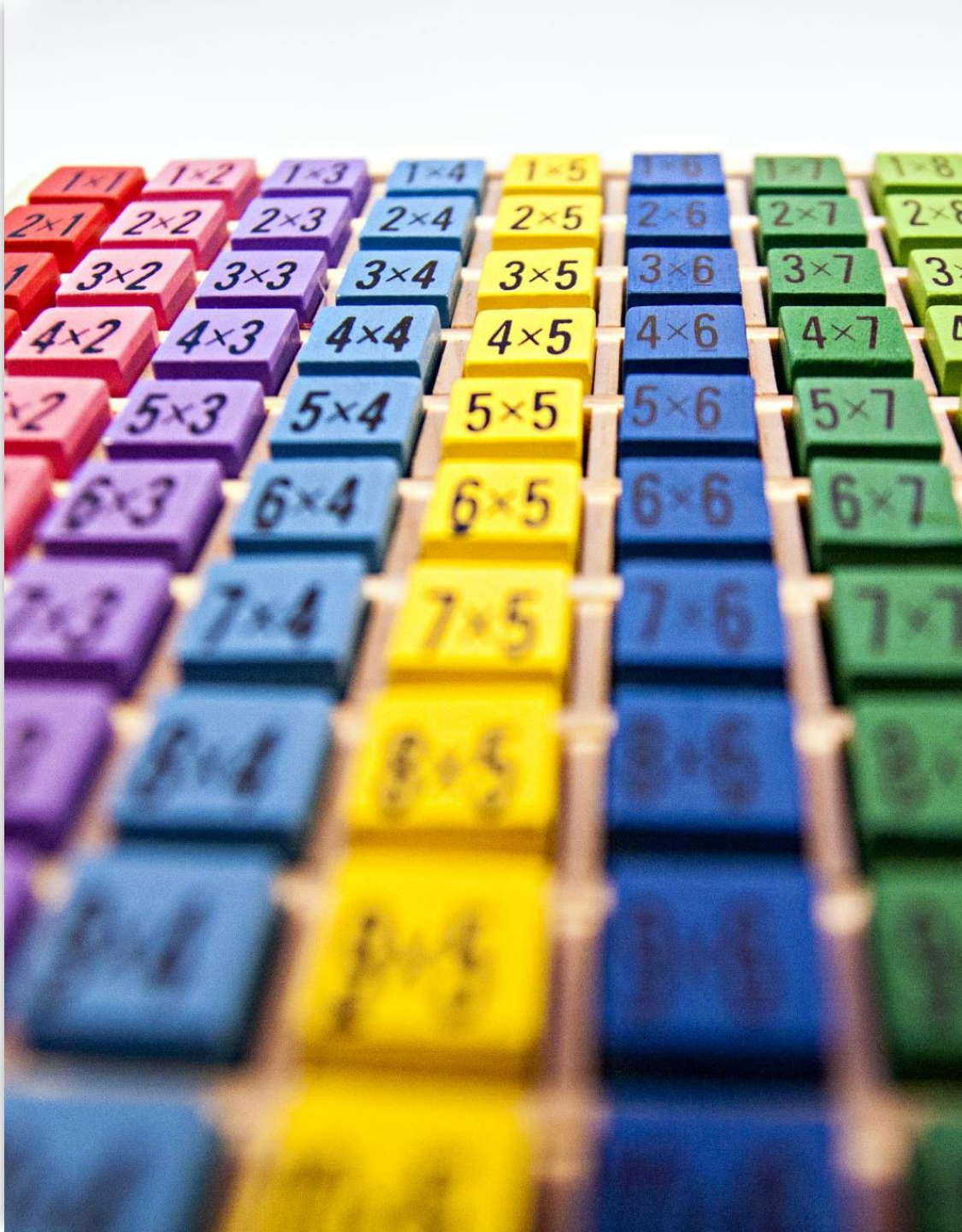
What is Classification

Classification in machine learning is a type of **supervised learning.**

The task of **classification** involves building a **model** that can **predict the category (or class)** of **new unseen data** points based on the trained data.

Examples of Classification Problems:

- **Binary Classification:** Involves **two classes** in the **class label(Dependent)**
  - **Spam Detection**: Classifying an email as either **"spam" or "not spam."**
  - **Disease Detection**: Predicting whether a patient has a disease **(e.g., cancer: "positive" or "negative").**
- **Multiclass Classification**: Involves more than two classes.
  - **Image Recognition**: Identifying objects in an image (**e.g., dog, cat, or bird**).



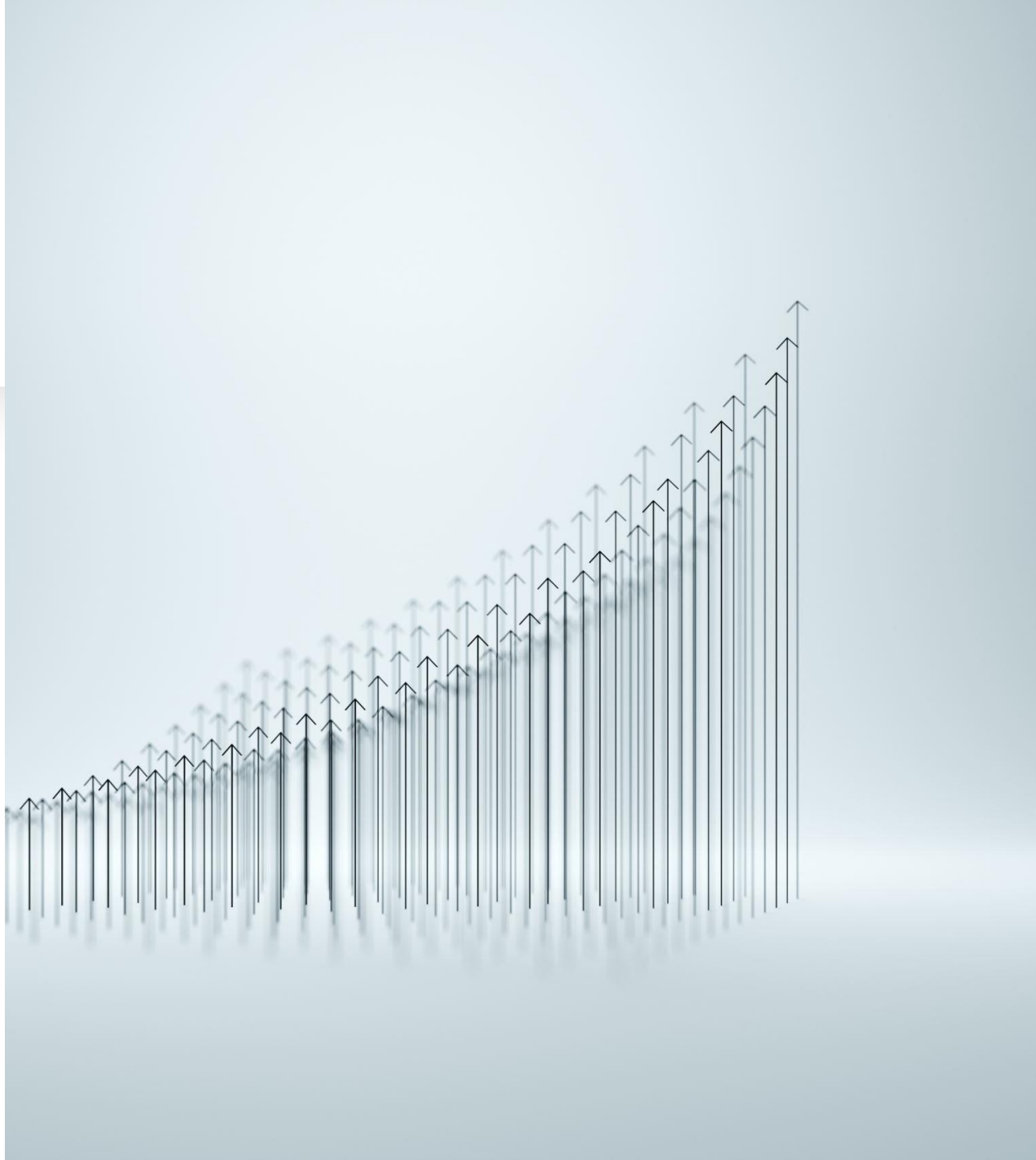
Types of Classification Algorithms

- **Logistic Regression**
- **Decision Trees**
- **Random Forest**
- **Support Vector Machines (SVM**
- **k-Nearest Neighbors (k-NN)**
- **Naive Bayes**
- **Neural Networks**

Difference between Classification and regression

classification, the output is a **discrete label or category (e.g., spam vs. not spam).**

Regression, the output is a **continuous value (e.g., predicting house prices).**



$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Positive and Negative Class

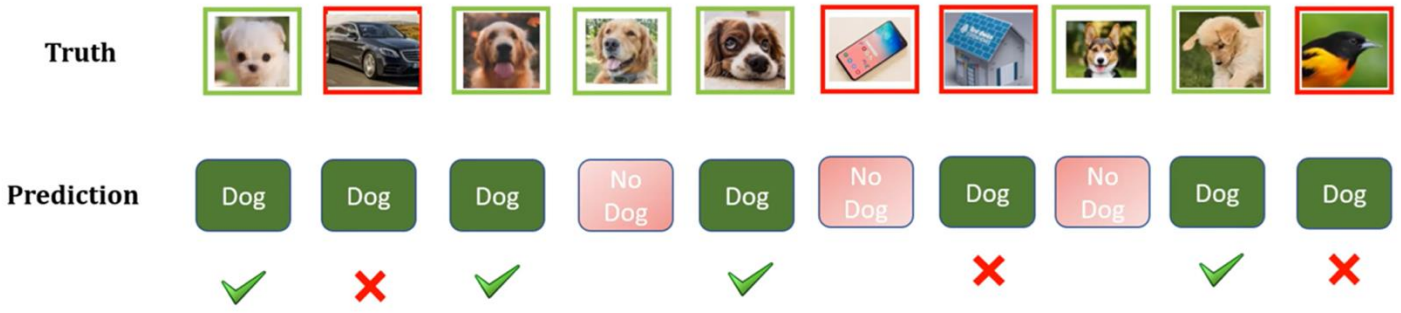
Accuracy is the **metric of prediction** that our model got for predicting the right results.

Accuracy can be formulated as follows:

Positive and Negative Class

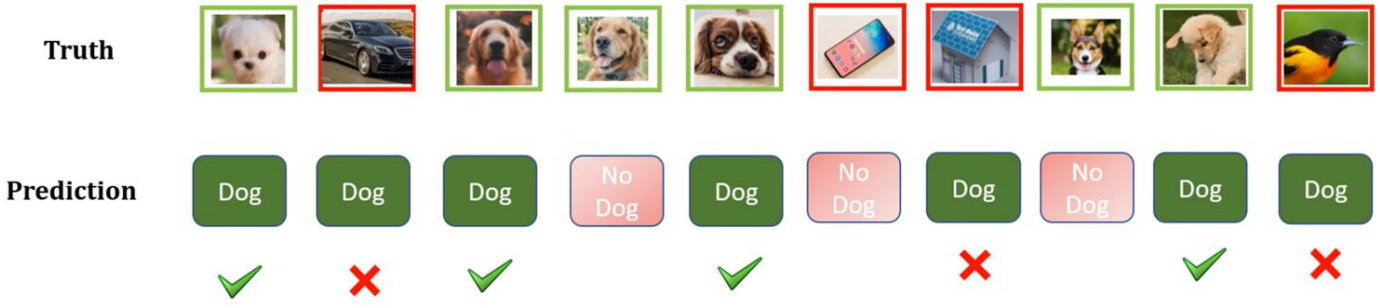
- **True Positive (TP):** is the result that we get if the model **correctly predict** the **positive class**
- **False Positive (FP):** is the result that we get if the model **predict a negative class as a positive class**
- **True Negative (TN):** is the result that we get if the model **correctly predict the negative class**
- **False Negative (FN):** is the result that we get if the model **predict a positive class as a negative class**

Positive and Negative Class



True Positive = 4

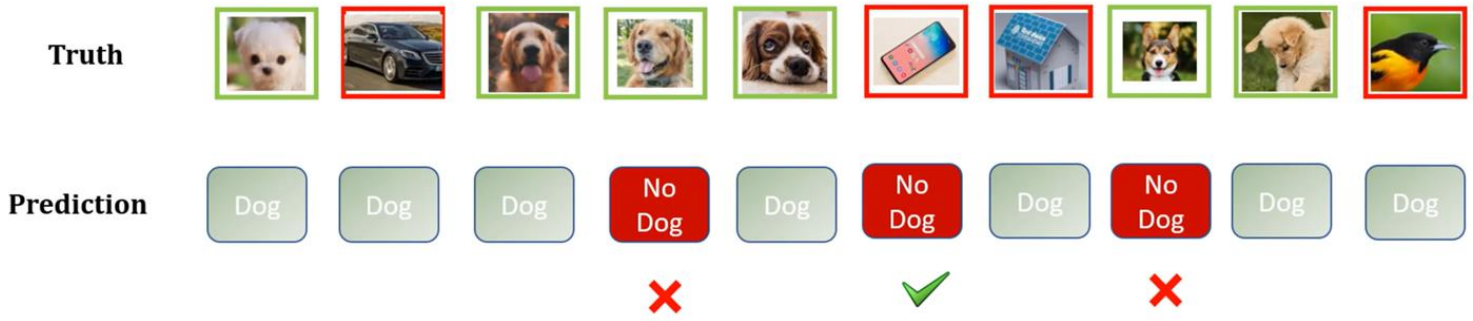
Positive and Negative Class



True Positive = 4

False Positive = 3

Positive and Negative Class



True Negative = 1

False Negative = 2

The Curse of Dimensionality

- The **Curse of Dimensionality** in Machine Learning happens when dealing with data that has **many features or dimensions**, making it harder for **algorithms** to **work effectively**.
- This can lead to problems like **increased computation time**, models that **overfit** the data **(perform well on training data but poorly on new data).**
- **Finding patterns** that aren't really meaningful.
- To overcome these challenges, techniques like **dimensionality reduction** and **data preprocessing** are essential to improve performance.

Problems Caused by Curse of Dimensionality

- **Increased Computation Time**
  - More features → more complex calculations.
  - Algorithms like k-NN, Decision Trees, or Neural Networks take longer to train.
- **Overfitting**
  - Models memorize training data instead of learning general patterns.
  - They perform well on training data but poorly on unseen (test) data.
- **Meaningless Patterns**
  - High dimensions may create random correlations that are not truly significant.
  - Example: In a dataset with thousands of features, the model may “think” feature X is important, but in reality, it’s just noise.

Dimensionality Reduction Techniques

Feature Selection:

- Feature selection involves identifying and choosing the **most relevant features** from the **original dataset**, while **discarding irrelevant or redundant ones**.
- This reduces the **dimensionality of the data**, making the model simpler and more efficient.

Example Dataset: Housing Prices

House ID	Square Footage	Number of Bedrooms	Number of Bathrooms	Distance to City Center (miles)	Age of House (years)	House Color	House Price ($)
1	2000	3	2	5	10	Red	250,000
2	1500	2	1	10	15	Blue	180,000
3	2500	4	3	2	5	Yellow	350,000
4	1800	3	2	8	20	Green	220,000
5	2200	4	2	6	8	Red	280,000

.....

Goal is to Identify the most relevant features for **predicting**

"House Price ($)“

After feature selection, you might find that **"Square Footage," "Number of Bedrooms," "Number of Bathrooms,"** and **"Distance to City Center"** and **"Age of House"** are the most important features for predicting the house price.

Features like and **"House Color"** might be considered less relevant and discarded.

Techniques for Feature Selection

- **Filter Methods** – Use statistical tests (e.g., correlation, Chi-square test).
- **Wrapper Methods** – Use ML models to test feature importance (e.g., Recursive Feature Elimination).
- **Embedded Methods** – Feature selection is built into algorithms (e.g., LASSO Regression, Decision Trees).

.....

Feature extraction:

Involves transforming the original **high-dimensional data** into a **lower-dimensional space** by creating **new features** that capture the essential information from the original data.

Techniques like **PCA(Principal Component Analysis)** are often used for this purpose.

Example:

Using the same house price dataset, if you apply **PCA**, it might combine multiple features like**(size, age, and number of rooms)** into a new feature that represents the **overall size and condition** of the house.

…

Data Pre processing

1. Normalization:

Normalization is a technique used to scale numerical features to a similar range, typically between **0 and 1.**

Example: Imagine you have a dataset about cars with the following features:

Car ID	Horsepower	Weight (lbs)	Price ($)
1	150	3000	20,000
2	200	3500	25,000
3	180	3200	22,000

Min-Max Normalization: Rescale the features to a [0, 1] range using the formula:

$$\text{Normalized Value} = \frac{\text{Value} - \text{Min Value}}{\text{Max Value} - \text{Min Value}}$$

Applying this to the "Horsepower" feature:

- For Car 1: $\frac{150-150}{200-150} = 0$
- For Car 2: $\frac{200-150}{200-150} = 1$
- For Car 3: $\frac{180-150}{200-150} = 0.6$

Car ID	Normalized Horsepower	Normalized Weight	Normalized Price
1	0.00	0.00	0.00
2	1.00	1.00	1.00
3	0.60	0.40	0.40

Now, all features are on a **comparable scale,** allowing the **algorithm** to consider **each feature equally.**

.....

2. Handling Missing Values:

Missing values occur when **no data** is available for certain observations in a dataset.

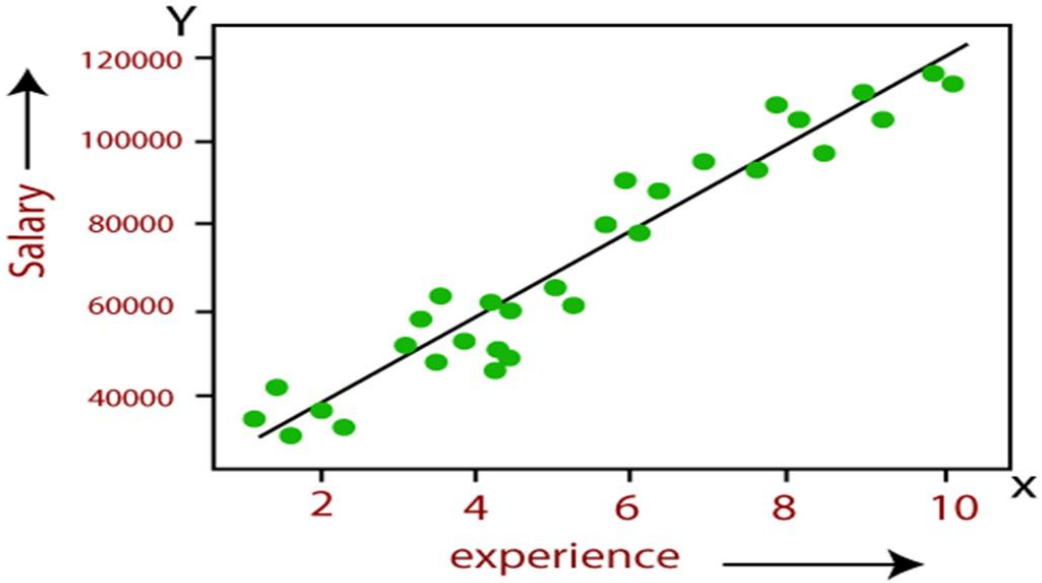
Handling **missing values** properly is crucial because they can lead to **biased models or errors** in analysis.

There are two common methods to handle missing data: **imputation and deletion.**

Linear Regression

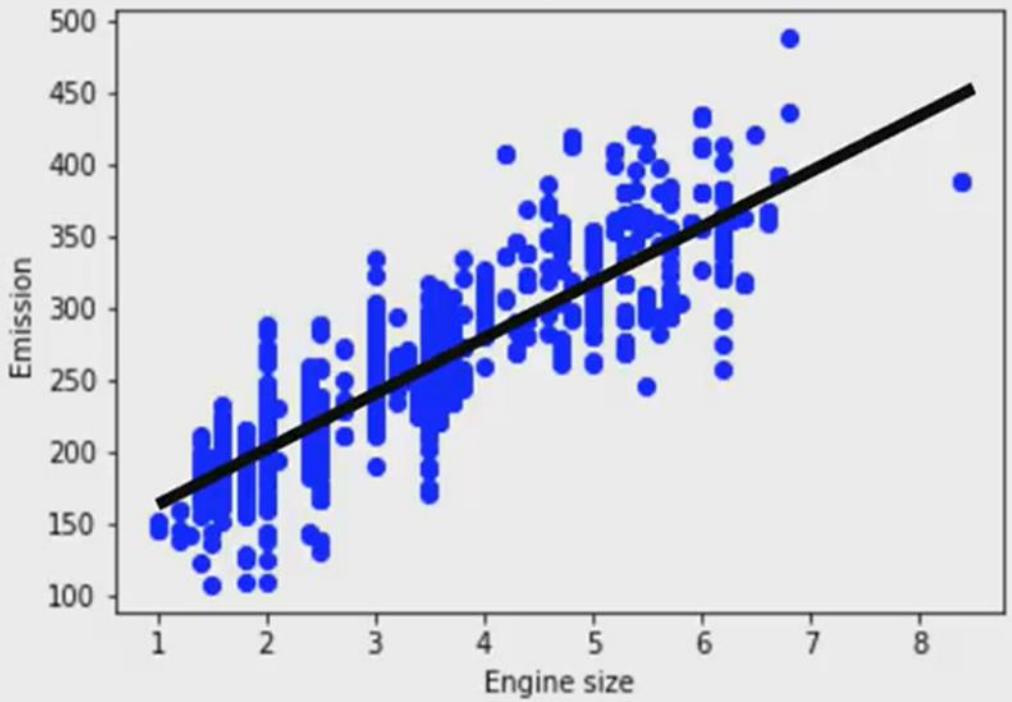
Linear regression is a **statistical method** used for **predictive analysis** that demonstrates a **linear relationship** between the **independent variable (X-axis)** and the **dependent variable (Y-axis), hence** it's referred to as **linear regression**.

Years of Experience	Expected Salary (in $)
1	40,000
2	45,000
3	50,000
4	55,000
5	60,000
6	65,000
7	70,000
8	75,000
9	80,000
10	90,000



How does linear regression work?

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



- With linear regression you can create a **Best fit line** through the data.
- And this **best fit line** will actually help us to do the **prediction**

X: Independent variable

Y: Dependent variable

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

Continuous Values

The key point in the **linear regression** is that our **dependent value** should be **continuous** and cannot be a discreet value.
However, the **independent variable(s)** can be measured on either a **categorical or continuous** measurement scale.

Types of Linear Regression

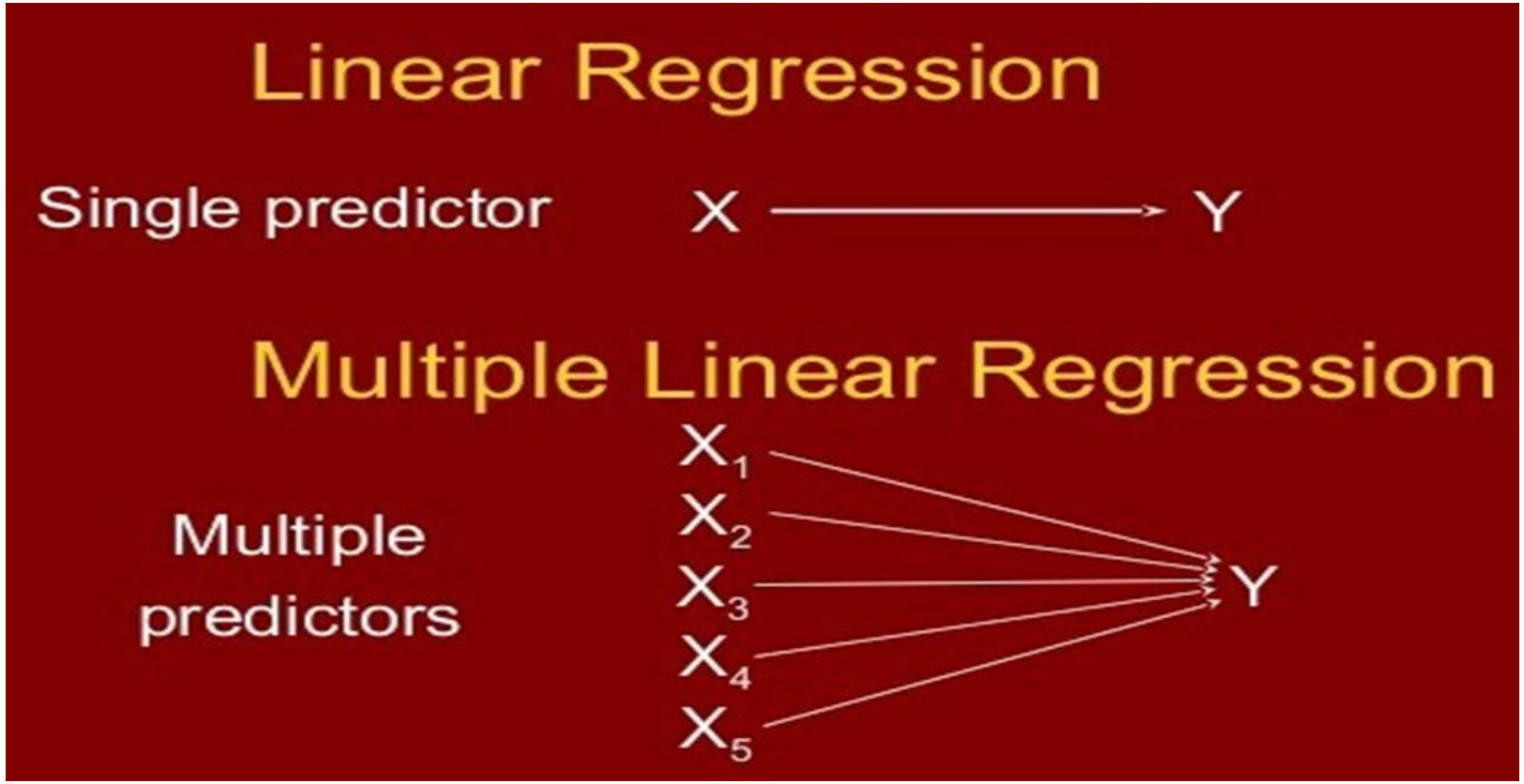
Simple Linear regression:

Simple linear regression models the **relationship between one independent variable (predictor)** and a **dependent variable (outcome).**

Multiple Linear regression:

Multiple linear regression models the **relationship between two or more independent variables** and a **dependent variable**.

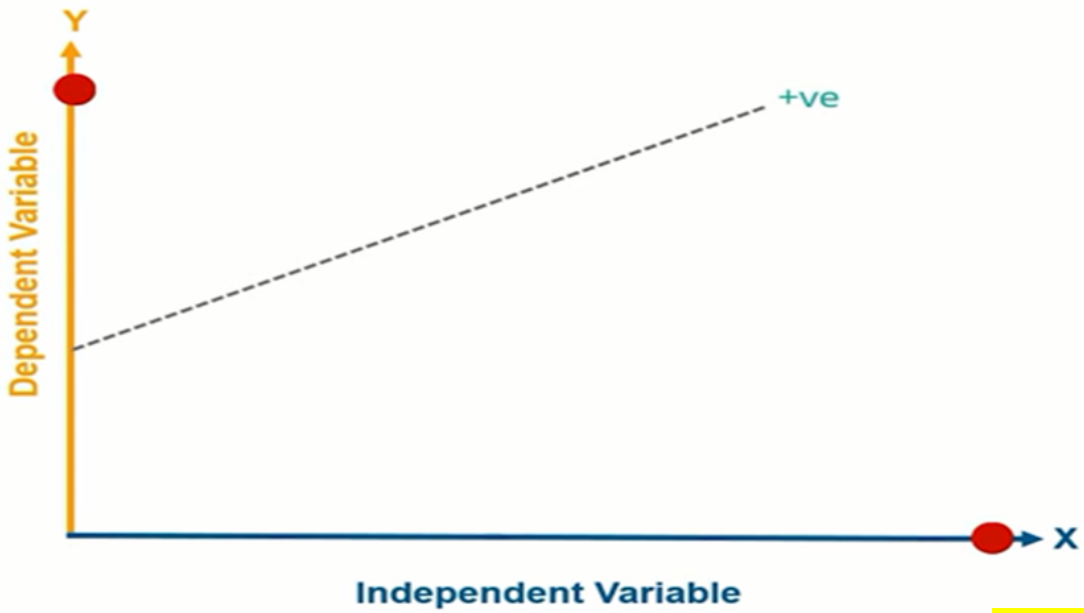
Simple vs Multiple



Applications of regression

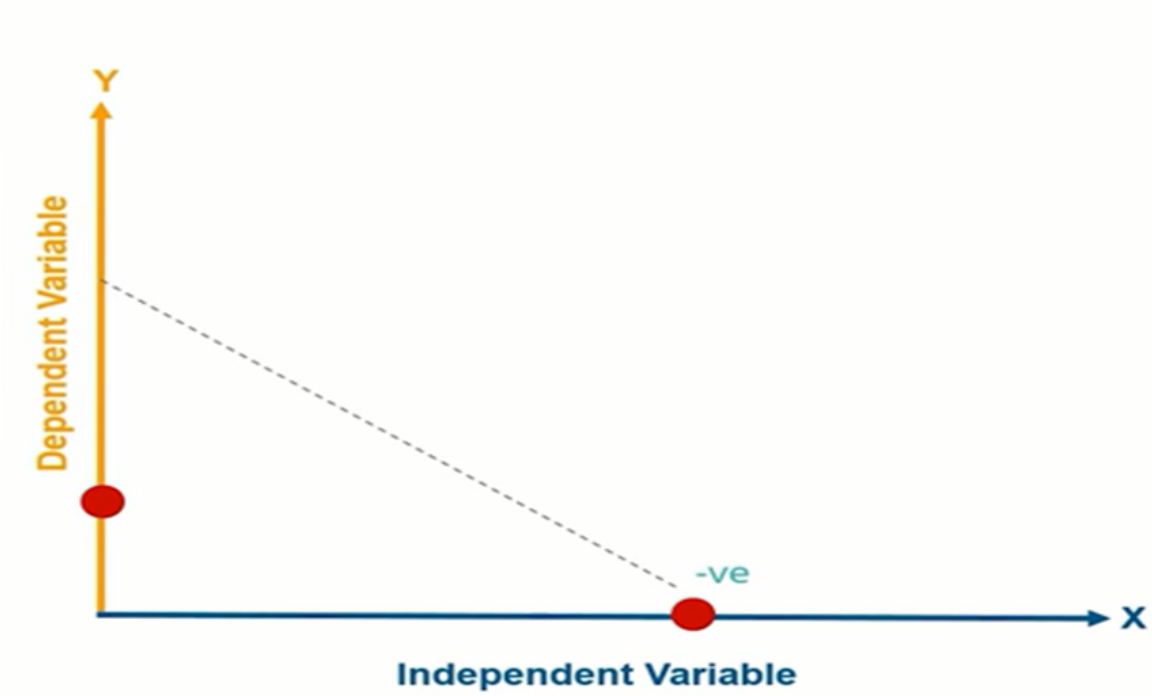
- Sales forecasting
- Satisfaction analysis
- Price estimation
- Employment income

Positive Linear Regression



As the **temperature increases**, **ice cream sales also increase**, showing a **positive linear relationship**.

Negative Linear Regression



As the **price of ice cream increases**, the **number of ice cream sales decreases**.

Simple Linear Regression Example

Sample Dataset

Independent Variable (x)	Dependent Variable(y)
1	2
2	4
3	5
4	4
5	5

Step1: You plot the data points on a graph with **Independent Variable (X)** on the X-axis and **Dependent Variable (Y)** on the **Y-axis.**

Step2: The goal of simple linear regression is to **fit a straight line** through **these points** that best represents the relationship between Years of Experience and Salary.

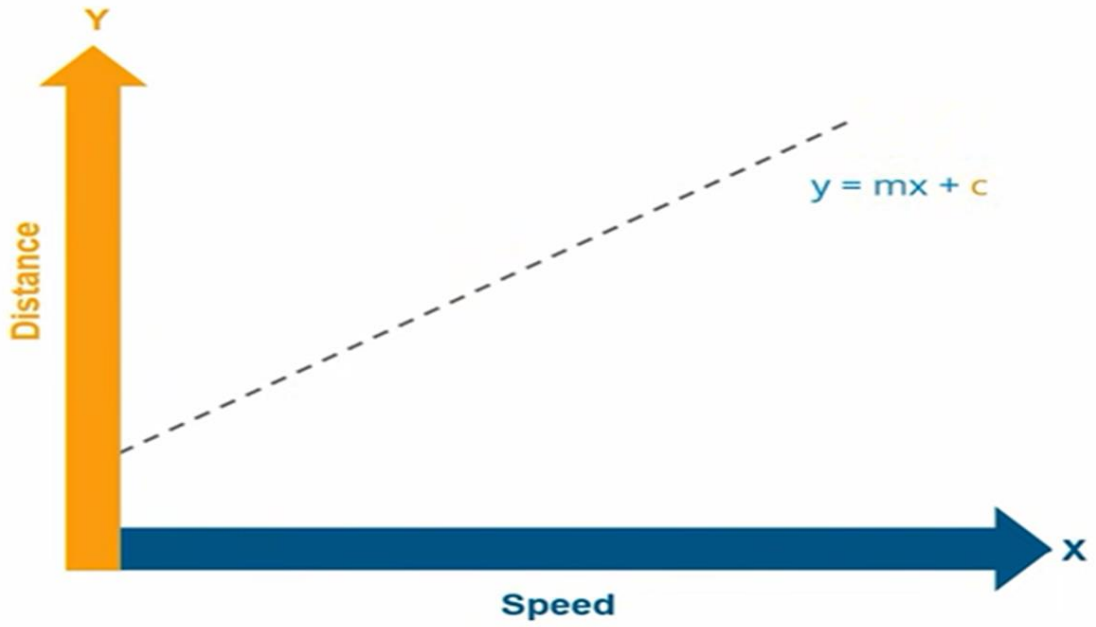
……

The line can be represented by the equation:

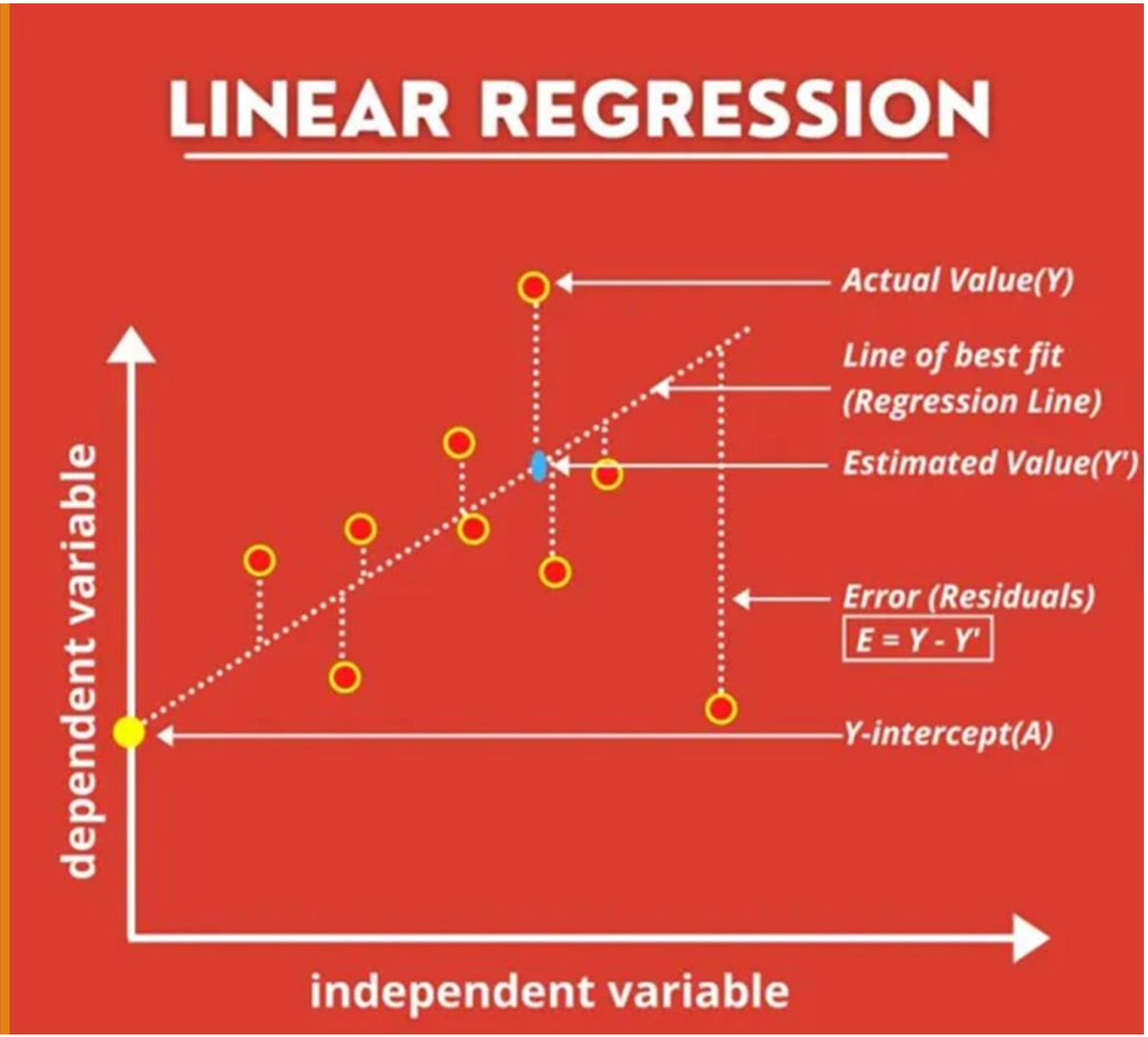
$$\hat{y} = b_0 + b_1 x$$

Where:

- $\hat{y}$ is the predicted salary (the outcome we want to predict).
- x is the years of experience (the input variable).
- b_1 is the slope of the line (it shows how much the salary increases for each additional year of experience).
- b_0 is the intercept (the starting salary when years of experience is 0).



Evaluation Metrics



Evaluation Metrics

In linear regression, several evaluation metrics are used to assess the performance of the model and understand how well it fits the data.

- **Mean Squared Error (MSE)**
- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**

Mean Absolute Error (MAE)

The average of the **absolute differences** between **predicted** and **actual values.**

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Where:

- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of data points

Lower MAE → The model makes **smaller errors on average** → **Better performance**.
Higher MAE → The model makes **larger errors on average** → **Worse performance**.

Mean Squared Error (MSE)

Mean Square Error (MSE) is a metric used to measure the **average** of the **squares of the differences** between the **predicted values** and the **actual values** in a dataset

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

When the **predicted values** are close to the **actual values**, the **differences (errors)** are **small**.

Smaller errors lead to a **smaller MSE**, meaning the **model's predictions** are **more accurate** overall.

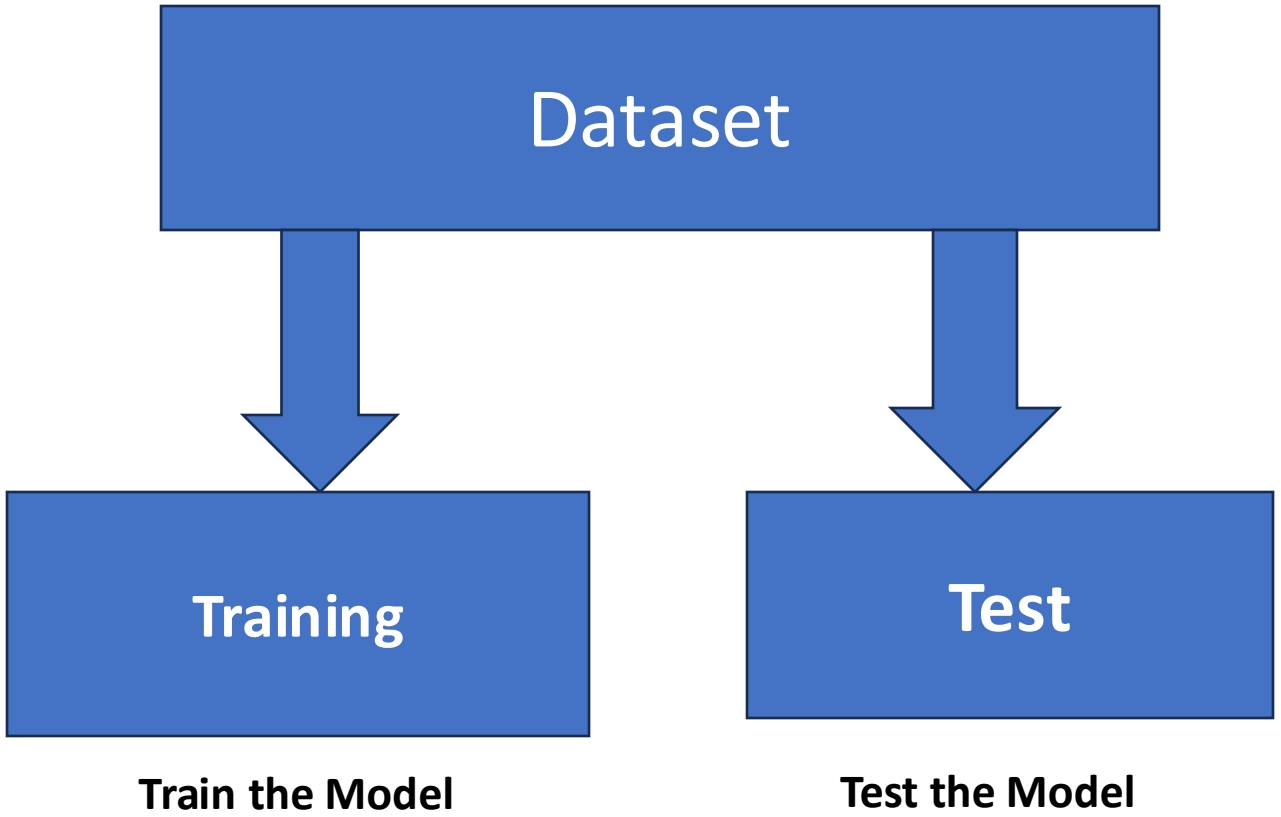
A **larger MSE** means the model's predictions are **far from the actual values**, indicating **worse performance**.

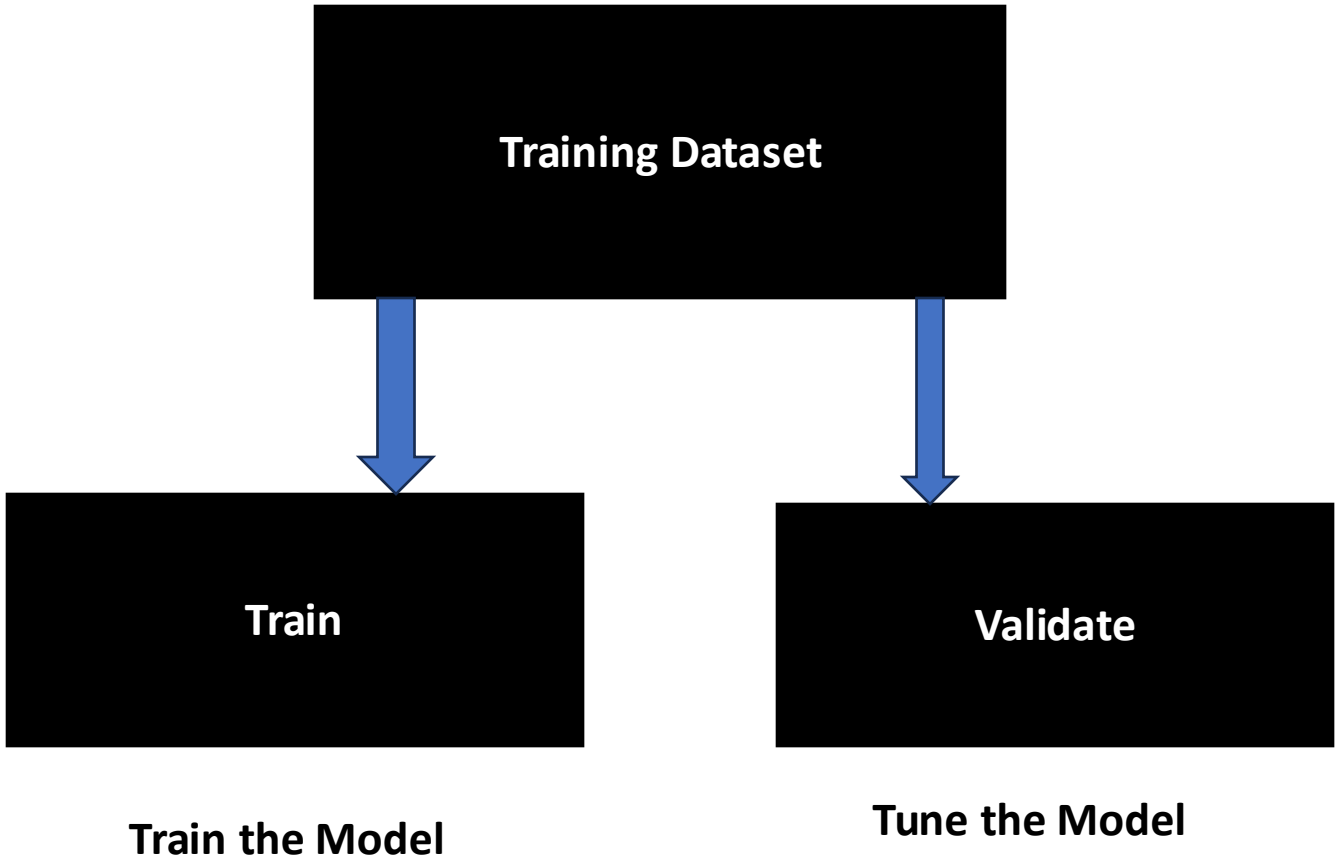
Root Mean Squared Error (RMSE)

- **MSE** gives you the **average squared error**, which is **hard to interpret** because it's in squared units.
- **RMSE** corrects this by **taking the square root.**
- **Making** the error **easier to interpret** since it's now in the same units as the target variable.

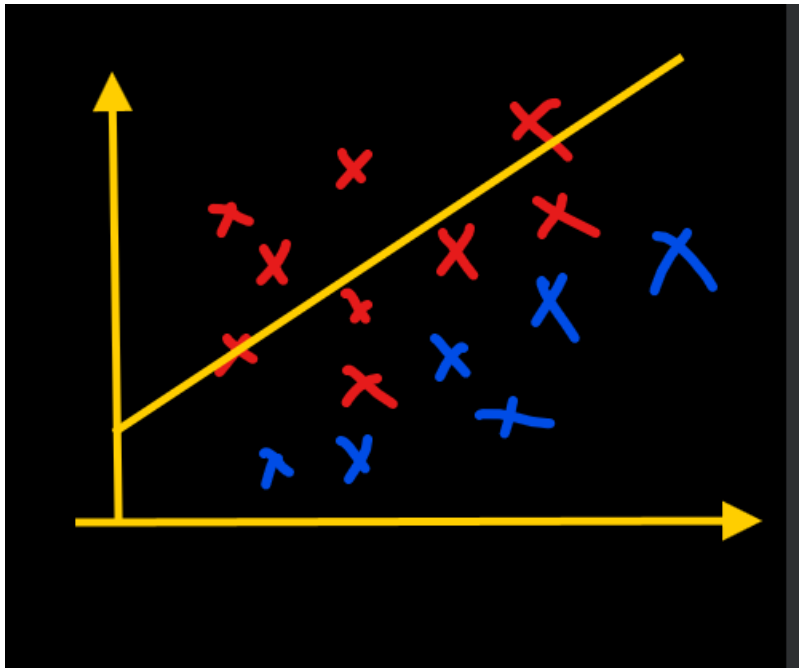
$$RMSE = \sqrt{MSE}$$

Overfitting and Underfitting




Training Dataset
Train
Validate
Train the Model
Tune the Model

. . . . . .



Model is Overfitting

Generalized Model

Train

Test

Very good Accuracy (90%)

Low Bias

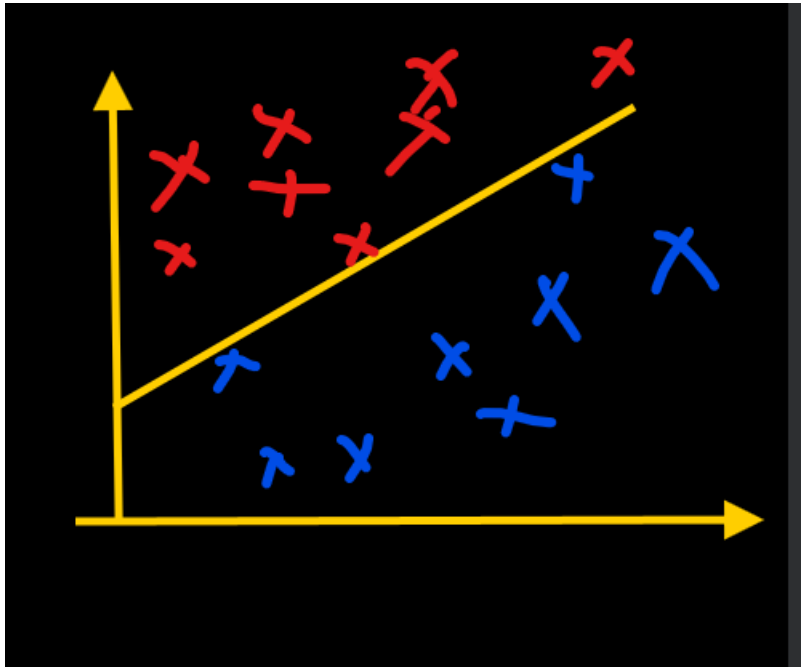
Very bad accuracy (50%)

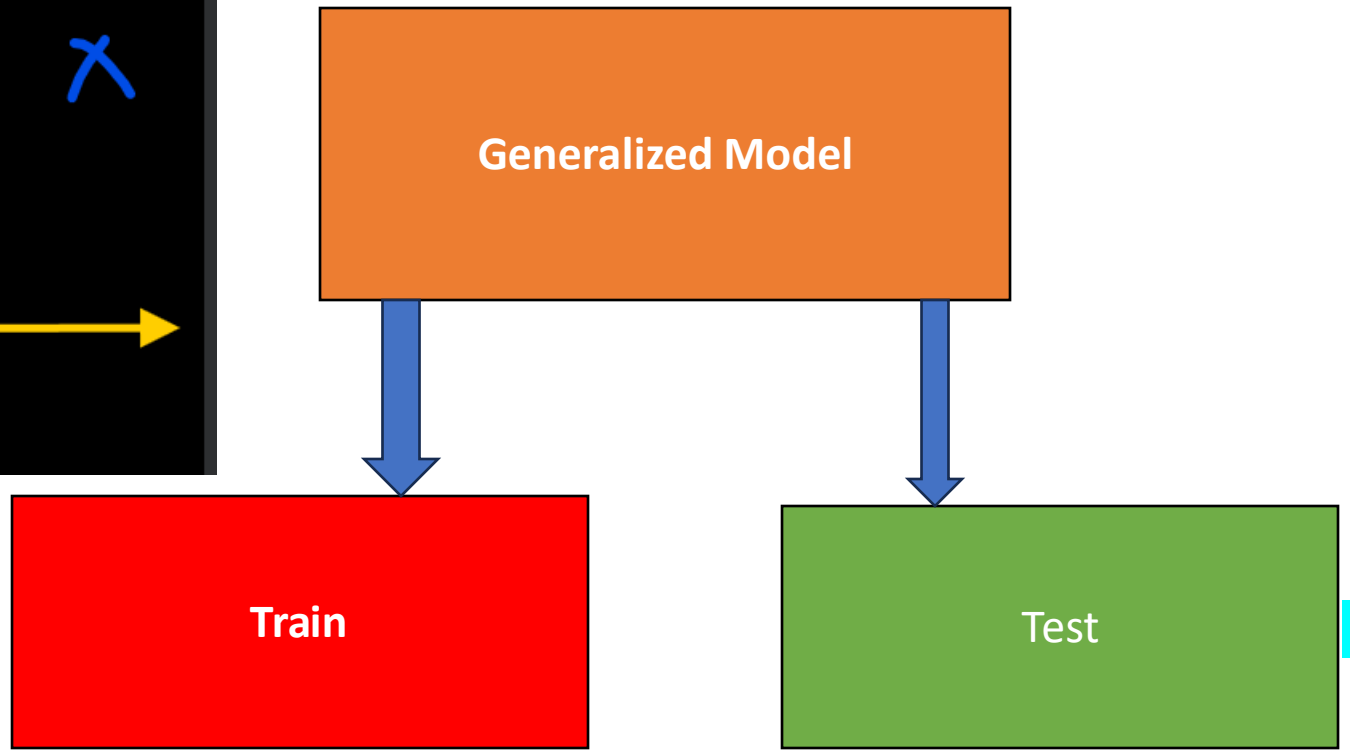
High Variance

Model **Trained well with Trained data**, but with respect to **test data model can't predict well** ,so it is **Overfitting**

. . . . . .

Model is Under fitting





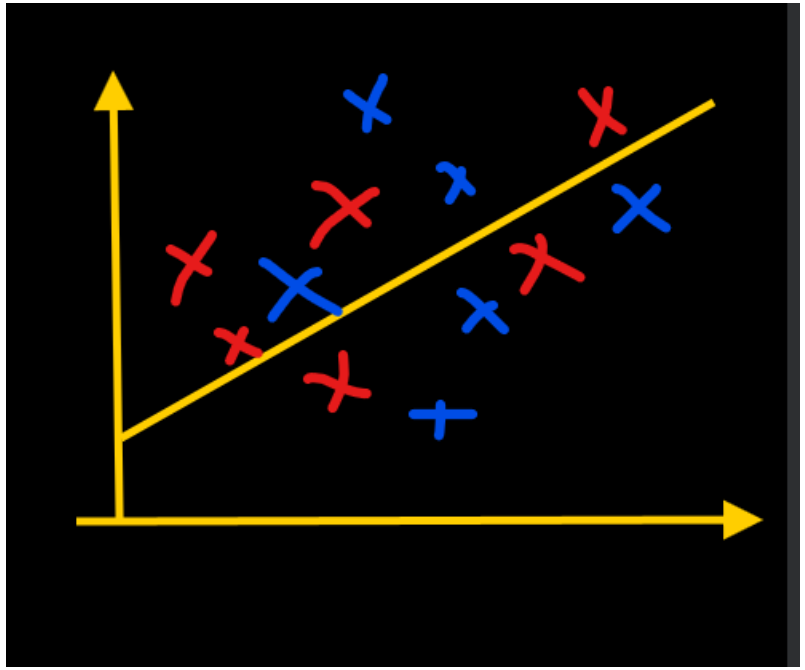
Accuracy is low (60%)

High Bias

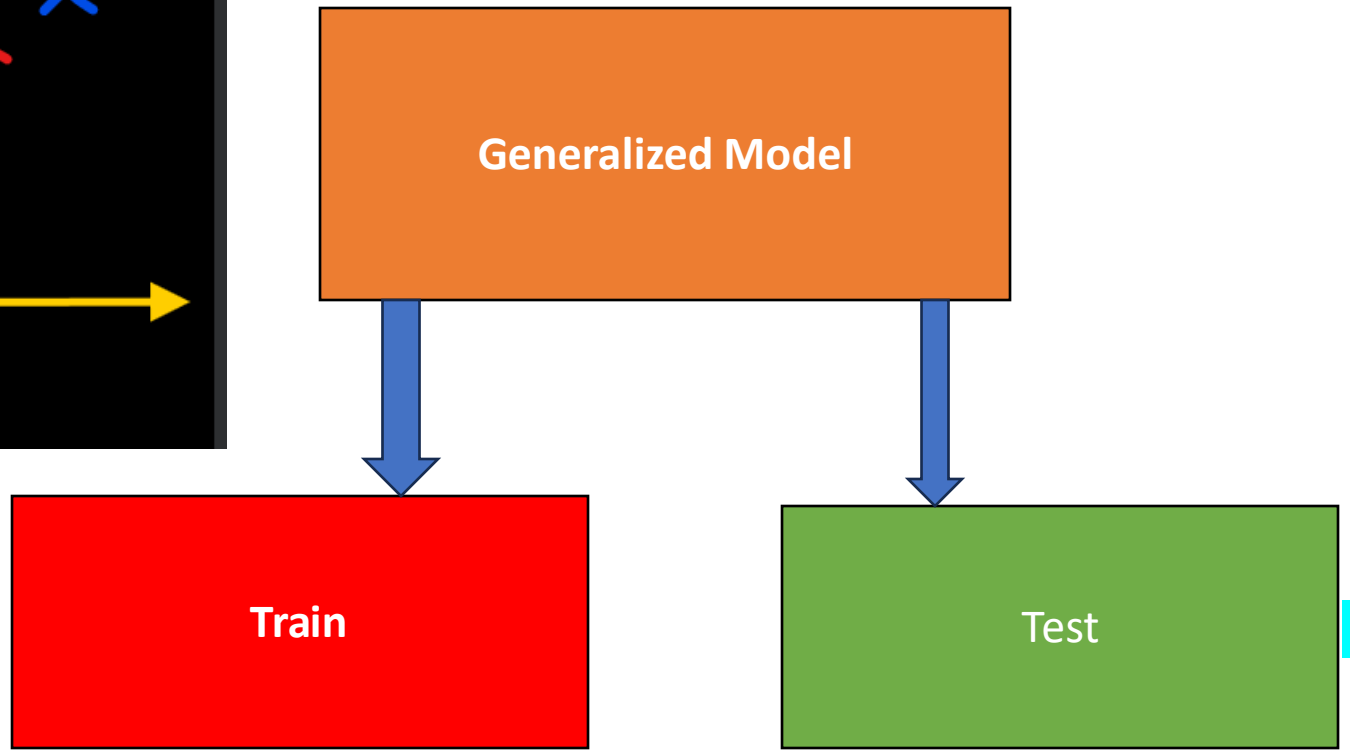
Accuracy is low (50%)

High Variance

Model poorly trained with Trained data, so **test data also can't predict well** ,so it is **Underfitting**



Model is Good fitting



Accuracy is High (90%)

Low Bias

Accuracy is high (85%)

Low Variance

Model trained well with the **Trained data**, **test data predict well with good accuracy** ,so it is **Good fitting**

Noise and Error

In machine learning, **noise and error** are important concepts that affect **model performance** and **understanding of the data** .

Noise:

It represents the **random errors or fluctuations** in the **data** that arise from various sources, such as **measurement errors**, **external factors, or inherent randomness.**

Characteristics

- **Unpredictable**

  Noise is typically **unpredictable** and does not follow any **specific pattern** that the model can learn.

- **Irrelevant to Prediction**:

  Noise does not provide **useful information** for the **prediction task** and can **ambiguous** the true relationship between the **features and the target variable.**

……

Error :

Error refers to the **difference between** the **predicted values** produced by the model and the **actual values** from the dataset.

It is a **measure** of how well or poorly the model is performing.

Types of error:

Bias: High bias indicates the model is **too simple** and may lead to **underfitting.**

Variance: High variance indicates the model is **too complex** and may lead to **overfitting**

Loss Function

- A **loss function** **measures** how well a **model's predictions** match the **actual target values.**
- The goal of **training a machine learning model** is to **minimize this loss function,** which in turn improves the **model's performance**.
- During training, **optimization algorithms** like **gradient descent** adjust the **model parameters** to **minimize the loss function**.

Types of Loss Function

1. **Mean Squared Error (MSE)** — for Regression
2. **Cross-Entropy Loss (Log Loss)** — for Classification
3. **Hinge Loss** — for Support Vector Machines (SVM)

Regularization

- Regularization in machine learning is a technique used to **prevent overfitting**, which occurs when a **model learns** not only the **underlying patterns in the training** data but also the **noise or irrelevant details**.
- Overfitting can lead to **poor generalization** performance on **unseen data**.
- Regularization **adds a penalty** to the **loss function** used to **train the model,** In order to **discourage the model** from becoming **too complex.**
- The idea is to **balance fitting the data well** with **keeping the model simple** to ensure it generalizes well to new data.

..

There are several types of regularization:

L1 Regularization (Lasso) **-Least Absolute Shrinkage and Selection Operator:** Adds a penalty to the **absolute value** of the **coefficients**.

$$\text{L1 penalty:} \quad \lambda \sum |w_i|$$

where w_i are the model coefficients and λ is the regularization strength.

During training, **L1 regularization** encourages the model to **shrink the less important feature weights to zero**. As a result, only the **most important features retain non-zero weights.**

This property makes **L1 regularization** useful for **feature selection.**

.....

L2 regularization(RIDGE): L2 regularization adds a penalty to the loss function based on the **squared values** of the model's weights.

Instead of setting weights to **exactly zero** **(as in L1 regularization),** L2 regularization **shrinks all weights** by a proportional amount, ensuring that they **remain small but non-zero**.

$$\text{L2 penalty:} \quad \lambda \sum w_i^2$$

Elastic Net: Combines both L1 and L2 regularization. It strikes a balance between feature selection and shrinking the coefficients.

$$\text{Elastic Net penalty:} \quad \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

By introducing **regularization**, **models** are **less likely** to **memorize the training data** and are **more likely** to perform **better on unseen data**.

Parametric Vs Non Parametric Model

Sno	Parametric Model	Non Parametric Model
1	These model has **strong assumption** to the dataset **follows a specific pattern**, like a straight line for linear regression.	These models don't have **any strong assumption** , it **won't follow a specific pattern**. They can adjust to fit the data, whatever shape it takes.
2	They use a **fixed number of parameters**, even if the given data size increases.	The number of **parameters grows** as you **add more data**, allowing the model to get more complex.
3	Since they have **fewer parameters to estimate**, they are **faster to train** and **easier to interpret**.	They can handle **more complex data**, but they need **more data and time** to train well.
4	If the data **doesn't fit** their **assumed pattern**, they may not work well. Example: **Linear regression**, where you assume data fits a straight line.	There is no **predefined assumption to any pattern**. Example: **k-Nearest Neighbors (k-NN**), which uses the data itself to make predictions without assuming any fixed pattern.
5	Can perform well with **small datasets**	Often requires **more data** to perform well. They can sometimes **fit** the **training data too well**, especially if the **data is small**.
6	**Limited flexibility**, may **underfit complex data**	**Highly flexible**, may **overfit small data**

Gradient Descent Curve

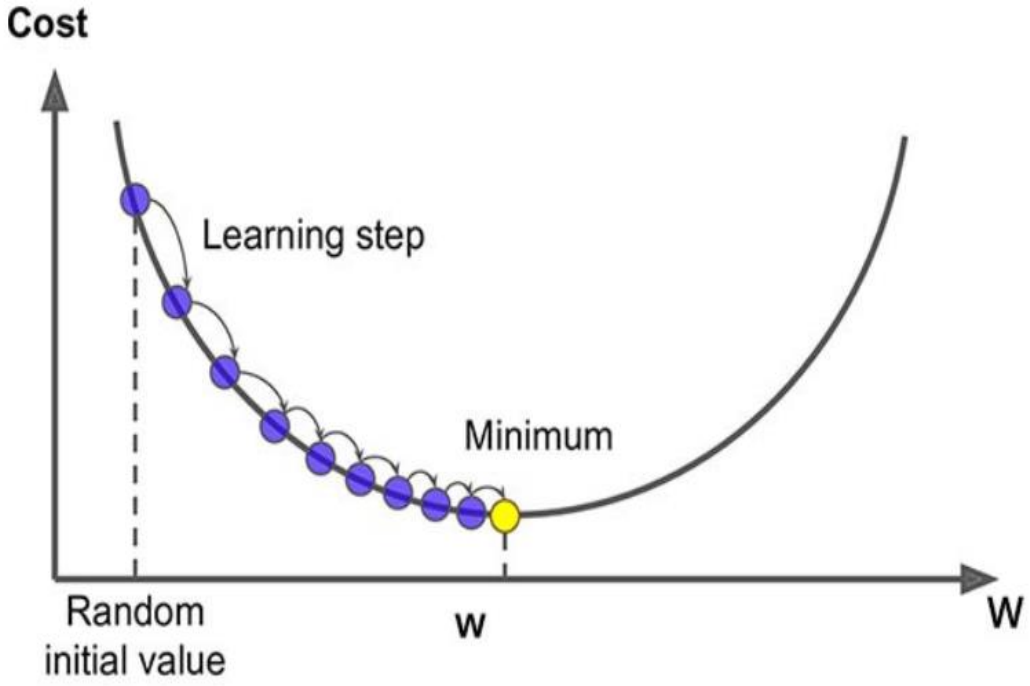
Gradient Descent is an **optimization algorithm** used to **minimize** the **loss function** in machine learning models.

The idea is to **iteratively adjust** the **model parameters** **weights** to find the **minimum of the loss function**, which corresponds to the **best model fit.**

1.Initial Loss (High):

- At the beginning of training, the **model's predictions** are usually **far** from the **true** values, so the initial loss value is high.
- On the curve, this appears as a **Random point** on the **y-axis** with a **high value**.

The curve illustrates how the **loss decreases** as the **model learns.**



loss function value on the **y-axis** and the **number of iterations** on the **x-axis**

.....

2. Descent (Learning Phase):

- As the algorithm **updates** the **model parameters** using **gradient descent**, the loss gradually decreases.
- On the curve, you'll see the line **descending**, indicating that **the model is learning and improving its predictions.**

3.Convergence (Flattening):

As the **gradient descent algorithm** gets closer to the **optimal solution (the global minimum of the loss function),** the **updates** to the **parameters** become smaller, and the **curve begins to flatten**.

…….

4. Final Loss (Minimum):

If the algorithm **successfully finds the global minimum**, the **loss reaches its lowest point.**